

Reviewer Pack — Minimal Frobenius Representation Count and Communication Com...

Entry 1216f0e17849 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 08:30:24 UTC

Minimal Frobenius Representation Count and Communication Complexity Rank Variance

Entry ID: 1216f0e17849 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 08:30:24 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Frobenius Representations) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every n -vertex bipartite graph G , the minimal number of distinct Frobenius representations required to encode any communication protocol for G is linearly correlated with its communication complexity rank variance, such that $|\text{Frobenius}(G)| = \Theta(\text{CommRankVar}(G))$

Rationale (proposer's reasoning):

Frobenius representations provide a way to encode non-commutative operations, which might capture hidden structures in communication protocols. A correlation between these representations and communication complexity rank variance could reveal insights into the intrinsic difficulty of communication tasks.

Taxonomy category: CONSTRUCTIVE_REPRESENTATION (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 05c9d554267db833

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each n -vertex bipartite graph G with $n \leq 40$, if $|\text{Frobenius}(G)| / \text{CommRankVar}(G)$ is within a threshold of 1 ± 0.05 for at least 80% of the graphs, the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Frobenius Representations" AND "communication complexity" AND matrix rank - "minimal Frobenius representations" AND communication complexity AND rank variance - bipartite graph AND Frobenius representation AND communication protocol complexity

Top relevant hits considered: - [http://arxiv.org/abs/0911.3482v5] Complexity of Networks (reprise) - [http://arxiv.org/abs/1709.04262v2] Lower Bounds for Approximating Graph Parameters via Communication Complexity - [http://arxiv.org/abs/1608.08324v2] Topological Drawings of Complete Bipartite Graphs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_bipartite_graph(n):
        A = [random.choice([0, 1]) for _ in range(n * n // 4)]
        B = [random.choice([0, 1]) for _ in range(n * n // 4)]
        adj_matrix = [[A[i * n + j] if i < n // 2 else B[(i - n // 2) * n + j] for j in range(n)]
        return adj_matrix

    def frobenius_representation_count(adj_matrix):
        n = len(adj_matrix)
        representations = set()
        for i, j in combinations(range(n), 2):
            if adj_matrix[i][j]:
                representations.add((i, j))
        return len(representations)

    def communication_complexity_rank_variance(adj_matrix):
        n = len(adj_matrix)
        rank = sum(sum(row) for row in adj_matrix) / (n * n)
        variance = sum((sum(row) - rank) ** 2 for row in adj_matrix) / (n * n)
        return variance

    n_max = 40
    instances_tested = 30
```

```

metric_values = []

for _ in range(instances_tested):
    n = random.randint(5, n_max)
    adj_matrix = generate_bipartite_graph(n)
    frobenius_count = frobenius_representation_count(adj_matrix)
    comm_rank_variance = communication_complexity_rank_variance(adj_matrix)

    if comm_rank_variance == 0:
        continue

    metric_values.append(frobenius_count / comm_rank_variance)

if not metric_values:
    return {
        "metric_name": "Frobenius(G) / CommRankVar(G)",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "communication_complexity_rank_variance_zero"
    }

mean = sum(metric_values) / len(metric_values)
std_dev = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
support_fraction = sum(1 for v in metric_values if abs(v - mean) <= 0.05 * mean) / len(metric_values)

return {
    "metric_name": "Frobenius(G) / CommRankVar(G)",
    "metric_value": mean,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": support_fraction >= 0.8,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_dev_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_dev_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"not supported\" first_failing_seed={first_failing_seed}")
    else:

```

```
print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
```

```
Traceback (most recent call last):
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_71cc408d.py", line 84, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_71cc408d.py", line 46, in run_trial
    adj_matrix = generate_bipartite_graph(n)
                ^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_71cc408d.py", line 23, in generate_bipa
    adj_matrix = [[A[i * n + j] if i < n // 2 else B[(i - n // 2) * n + j] for j in range(n)] for i in range(
                ~^^^^^^^^^^^^
```

```
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's validity according to the pre-registered support condition. | next: Review and debug the test code to ensure it can run without crashing and produce the necessary data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13719
2	preregistration	ollama_remote	glm4:latest	0	0	9518
3	novelty	ollama_remote	glm4:latest	0	0	8417
4	novelty	ollama_remote	glm4:latest	0	0	9733
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	47418
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11159
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14849
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12157
9	judge	ollama_remote	glm4:latest	0	0	13536

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 140506 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/1216f0e17849.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/1216f0e17849.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/1216f0e17849.tar.gz (if generated)